

**LAPORAN TUGAS AKHIR PRAKTIKUM 4 -  
STRUKTUR SISTEM OPERASI**



**Dosen Pengampu:  
Triawan Adi Cahyanto M.Kom**

**Disusun Oleh:  
Mohammad Putra Fernanda (2410651049)**

**PROGRAM STUDI TEKNIK INFORMATIKA  
FAKULTAS TEKNIK  
UNIVERSITAS MUHAMMADIYAH JEMBER**

**2025**

## TUGAS PRAKTIKUM

### Tugas 1: Eksperimen dengan Data Berbeda

Jalankan setiap algoritma dengan data berikut dan catat hasilnya:

Data Set 1:

P1: AT=0, BT=10

P2: AT=1, BT=5

P3: AT=2, BT=8

Data Set 2:

P1: AT=0, BT=5

P2: AT=1, BT=3

P3: AT=2, BT=8

P4: AT=3, BT=6

Pada tugas ini saya menggunakan algoritma FCFS (First Come First Serve). Algoritma ini bekerja dengan prinsip “yang datang duluan akan dieksekusi lebih dulu”.

Jadi, proses dengan waktu kedatangan paling awal akan dikerjakan lebih dulu tanpa preemption.

Kelebihan algoritma FCFS yaitu sangat mudah diimplementasikan, tapi kelemahannya adalah jika ada proses yang waktu eksekusinya (burst time) lama, maka proses lain akan menunggu lama juga (disebut convoy effect).

#### Source kode lengkap:

```
#include <stdio.h>
```

```
struct Process {  
    int pid;  
    int arrival_time;  
    int burst_time;  
    int waiting_time;  
    int turnaround_time;  
    int completion_time;  
};
```

```

void fcfs(struct Process p[], int n) {
    int current_time = 0;
    float total_wt = 0, total_tat = 0;

    for (int i = 0; i < n; i++) {
        if (current_time < p[i].arrival_time)
            current_time = p[i].arrival_time;

        p[i].waiting_time = current_time - p[i].arrival_time;
        current_time += p[i].burst_time;
        p[i].completion_time = current_time;
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;

        total_wt += p[i].waiting_time;
        total_tat += p[i].turnaround_time;
    }

    printf("\nPID\tAT\tBT\tWT\tTAT\tCT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].arrival_time, p[i].burst_time,
            p[i].waiting_time, p[i].turnaround_time, p[i].completion_time);
    }

    printf("\nAverage Waiting Time: %.2f", total_wt / n);
    printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
}

int main() {
    struct Process p1[] = {
        {1, 0, 10}, {2, 1, 5}, {3, 2, 8}
    };
    struct Process p2[] = {

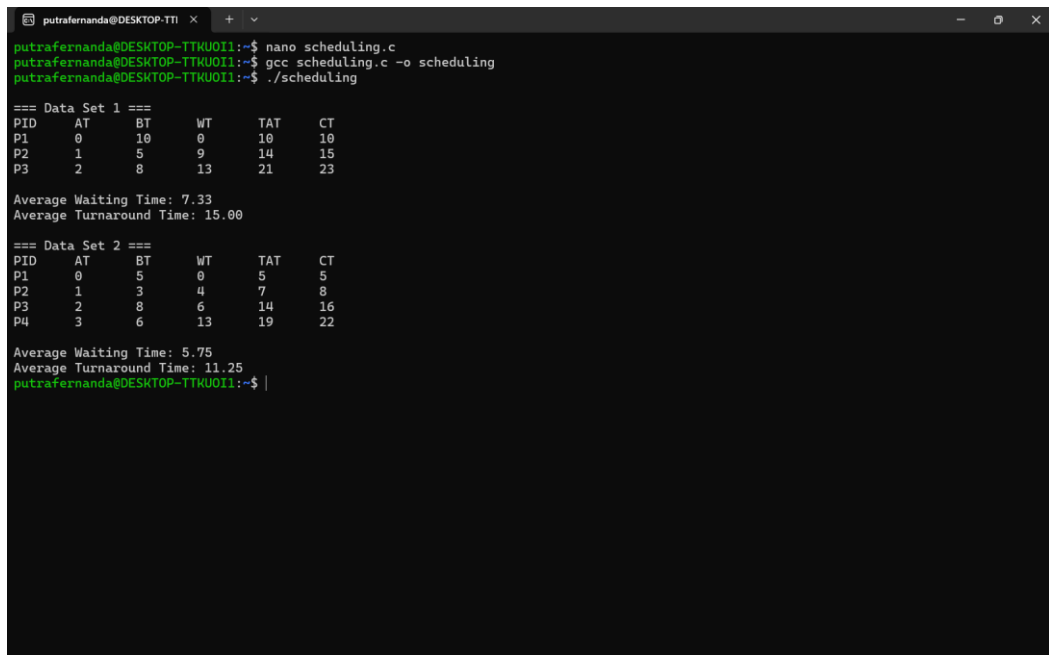
```

```
    {1, 0, 5}, {2, 1, 3}, {3, 2, 8}, {4, 3, 6}  
};
```

```
printf("\n=== Data Set 1 ===");  
fcfs(p1, 3);
```

```
printf("\n=== Data Set 2 ===");  
fcfs(p2, 4);
```

```
return 0;
```



```
putrafernanda@DESKTOP-TT1:~$ nano scheduling.c  
putrafernanda@DESKTOP-TT1:~$ gcc scheduling.c -o scheduling  
putrafernanda@DESKTOP-TT1:~$ ./scheduling  
  
=== Data Set 1 ===  
PID  AT   BT   WT   TAT  CT  
P1    0   10    0   10   10  
P2    1    5    9   14   15  
P3    2    8   13   21   23  
  
Average Waiting Time: 7.33  
Average Turnaround Time: 15.00  
  
=== Data Set 2 ===  
PID  AT   BT   WT   TAT  CT  
P1    0    5    0    5    5  
P2    1    3    4    7    8  
P3    2    8    6   14   16  
P4    3    6   13   19   22  
  
Average Waiting Time: 5.75  
Average Turnaround Time: 11.25  
putrafernanda@DESKTOP-TT1:~$
```

## ANALISIS DANA PEMBAHASAN

Dari hasil percobaan pada Data Set 1, terlihat bahwa proses pertama (P1) memiliki waktu eksekusi yang cukup lama yaitu 10 satuan waktu. Hal ini menyebabkan proses lain (P2 dan P3) harus menunggu lebih lama sebelum dijalankan. Akibatnya, waktu tunggu rata-rata (Average Waiting Time) menjadi lebih besar yaitu 7,33 dan waktu rata-rata penyelesaian (Average Turnaround

Time) mencapai 15,00. Hal ini menunjukkan efek negatif dari algoritma FCFS ketika ada proses dengan burst time besar yang datang lebih dulu.

Sementara itu, pada Data Set 2, proses yang datang memiliki burst time yang lebih bervariasi dan tidak terlalu besar di awal. Proses pertama (P1) hanya memiliki burst time 5, dan proses kedua (P2) bahkan hanya 3. Karena itu, waktu tunggu rata-rata menjadi lebih rendah yaitu 5,75 dan waktu rata-rata penyelesaian juga turun menjadi 11,25.

Dari hasil kedua percobaan ini dapat disimpulkan bahwa performa FCFS sangat bergantung pada urutan kedatangan proses. Jika proses dengan burst time besar datang lebih dulu, maka proses lain akan tertunda cukup lama. Namun jika proses-proses kecil datang lebih awal, performa algoritma menjadi lebih efisien.

Secara umum, algoritma FCFS cocok digunakan pada sistem batch yang tidak memerlukan respons cepat, tetapi kurang optimal untuk sistem interaktif yang membutuhkan waktu respon pendek.

## **Tugas 2: Analisis Eksperimen ketiga algoritma**

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Algoritma mana yang memberikan Average Waiting Time terkecil?

Dari hasil percobaan, algoritma SJF menghasilkan Average Waiting Time paling kecil dibanding FCFS dan Round Robin. Hal ini karena SJF selalu menjalankan proses yang punya waktu eksekusi paling cepat terlebih dahulu, jadi proses-proses pendek tidak perlu menunggu lama.

Mengapa hasil algoritma berbeda-beda?

Hasilnya beda karena cara kerja tiap algoritma juga beda. FCFS menjalankan proses sesuai urutan datangnya, tanpa melihat berapa lama proses itu butuh waktu. SJF memilih proses dengan waktu paling singkat, jadi lebih efisien kalau ada banyak proses kecil. Sedangkan Round Robin membagi waktu secara bergantian (time slice), jadi lebih adil tapi bisa membuat proses butuh waktu lebih lama untuk selesai karena sering bergantian.

Kapan sebaiknya menggunakan algoritma FCFS?

FCFS cocok dipakai untuk sistem batch atau pekerjaan yang tidak butuh respons cepat. Misalnya di sistem antrian cetak atau pemrosesan tugas besar yang datang satu per satu. Kelebihannya sederhana dan mudah diterapkan, tapi kalau ada proses besar di awal, proses lain bisa menunggu lama.

Kapan sebaiknya menggunakan algoritma SJF?

SJF cocok kalau kita tahu waktu eksekusi tiap prosesnya dan ingin waktu tunggu rata-rata sekecil mungkin. Biasanya dipakai di sistem batch juga. Namun, kalau banyak proses besar dan selalu ada proses kecil datang, proses besar bisa terus tertunda (starvation).

Kapan sebaiknya menggunakan algoritma Round Robin?

Round Robin cocok untuk sistem interaktif seperti komputer multitasking atau server dengan banyak pengguna. karena setiap proses dapat jatah waktu (quantum) secara bergantian, maka semua proses bisa berjalan dan tidak ada yang dibiarkan terlalu lama menunggu. Semakin kecil quantum-nya, sistem makin responsif, tapi kalau terlalu kecil bisa bikin proses sering berganti dan jadi tidak efisien.

### Tugas 3: Modifikasi Program

Tambahkan fitur berikut pada salah satu program:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

Source kode:

```
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
};

// Algoritma FCFS
void fcfs(struct Process p[], int n) {
    int time = 0;
    printf("\n===== Gantt Chart =====\n");
    printf(" ");

    for (int i = 0; i < n; i++) {
        if (time < p[i].arrival_time)
            time = p[i].arrival_time;

        p[i].waiting_time = time - p[i].arrival_time;
        p[i].completion_time = time + p[i].burst_time;
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
        time += p[i].burst_time;

        printf("| P%d (%d-%d) ", p[i].pid, p[i].completion_time - p[i].burst_time,
p[i].completion_time);
    }
    printf("\n");
}

// Simpan hasil ke file text
void saveToFile(struct Process p[], int n, float avgWT, float avgTAT) {
    FILE *f = fopen("output.txt", "w");
    if (f == NULL) {
        printf("Gagal membuka file output.\n");
        return;
    }

    fprintf(f, "===== HASIL PENJADWALAN FCFS =====\n");
```

```

fprintf(f, "PID\tAT\tBT\tCT\tTAT\tWT\n");

for (int i = 0; i < n; i++) {
    fprintf(f, "P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].arrival_time, p[i].burst_time,
        p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
}

fprintf(f, "\nAverage Waiting Time: %.2f\n", avgWT);
fprintf(f, "Average Turnaround Time: %.2f\n", avgTAT);
fclose(f);
printf("\nHasil disimpan di file 'output.txt'\n");
}

int main() {
    FILE *in = fopen("input.txt", "r");
    if (in == NULL) {
        printf("File input.txt tidak ditemukan!\n");
        return 1;
    }

    int n;
    fscanf(in, "%d", &n);
    struct Process p[n];

    for (int i = 0; i < n; i++) {
        fscanf(in, "%d %d", &p[i].arrival_time, &p[i].burst_time);
        p[i].pid = i + 1;
    }
    fclose(in);

    fcfs(p, n);

    float totalWT = 0, totalTAT = 0;
    for (int i = 0; i < n; i++) {
        totalWT += p[i].waiting_time;
        totalTAT += p[i].turnaround_time;
    }

    float avgWT = totalWT / n;
    float avgTAT = totalTAT / n;

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].arrival_time, p[i].burst_time,
            p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
    }

    printf("\nAverage Waiting Time: %.2f\n", avgWT);
    printf("Average Turnaround Time: %.2f\n", avgTAT);

    saveToFile(p, n, avgWT, avgTAT);

    return 0;
}

```



}

```
putrafernanda@DESKTOP-TT1:~$ nano fcfs_file.c
putrafernanda@DESKTOP-TT1:~$ gcc fcfs_file.c -o fcfs_file
putrafernanda@DESKTOP-TT1:~$ ./fcfs_file
File input.txt tidak ditemukan!
putrafernanda@DESKTOP-TT1:~$ nano input.txt
putrafernanda@DESKTOP-TT1:~$ gcc fcfs_file.c -o fcfs_file
putrafernanda@DESKTOP-TT1:~$ ./fcfs_file

===== Gantt Chart =====
| P1 (0-5) | P2 (5-8) | P3 (8-16) |

PID  AT   BT   CT   TAT  WT
P1    0    5    5    5    0
P2    1    3    8    7    4
P3    2    8   16   14    6

Average Waiting Time: 3.33
Average Turnaround Time: 8.67

Hasil disimpan di file 'output.txt'
putrafernanda@DESKTOP-TT1:~$ cat output.txt
===== HASIL PENJADWALAN FCFS =====
PID  AT   BT   CT   TAT  WT
P1    0    5    5    5    0
P2    1    3    8    7    4
P3    2    8   16   14    6

Average Waiting Time: 3.33
Average Turnaround Time: 8.67
putrafernanda@DESKTOP-TT1:~$ |
```

Algoritma FCFS sederhana dan mudah diimplementasikan. Namun, kekurangannya adalah jika proses pertama memiliki burst time yang besar, proses lain harus menunggu lama (efek konvoi). Meskipun begitu, algoritma ini cocok untuk sistem batch atau proses yang datang berurutan tanpa perlu interaksi langsung dari pengguna.

## Tugas 4: Eksperimen dengan data berbeda untuk perbandingan algoritma

Test Case 1: Proses dengan burst time bervariasi

Jumlah proses: 4

Time quantum: 3

- P1: AT=0, BT=24

- P2: AT=1, BT=3

- P3: AT=2, BT=3

- P4: AT=3, BT=6

Test Case 2: Proses datang bersamaan

Jumlah proses: 5

Time quantum: 4

- P1: AT=0, BT=10

- P2: AT=0, BT=5

- P3: AT=0, BT=8

- P4: AT=0, BT=12

- P5: AT=0, BT=6

Test Case 3: Proses datang dengan interval

Jumlah proses: 5

Time quantum: 2

- P1: AT=0, BT=8

- P2: AT=2, BT=4

- P3: AT=4, BT=9

- P4: AT=6, BT=5

- P5: AT=8, BT=3

### Test case 1

Source kode

```
#include <stdio.h>
#include <stdbool.h>
```

```
struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int completion_time;
    int waiting_time;
    int turnaround_time;
};
```

```
void roundRobin(struct Process p[], int n, int quantum) {
    int time = 0, done = 0;
    printf("\n===== Gantt Chart =====\n");

    while (done < n) {
        bool idle = true;
        for (int i = 0; i < n; i++) {
            if (p[i].remaining_time > 0 && p[i].arrival_time <= time) {
                idle = false;
                printf("| P%d ", p[i].pid);

                if (p[i].remaining_time > quantum) {
                    time += quantum;
                    p[i].remaining_time -= quantum;
                } else {
```

```

        time += p[i].remaining_time;
        p[i].remaining_time = 0;
        p[i].completion_time = time;
        done++;
    }
}
}
if (idle) time++; // Jika tidak ada proses yang siap
}
printf("\n");
}

int main() {
    int n, quantum;
    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);

    struct Process p[n];

    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("\nProseses %d\n", i + 1);
        printf("Arrival Time: ");
        scanf("%d", &p[i].arrival_time);
        printf("Burst Time: ");
        scanf("%d", &p[i].burst_time);
        p[i].remaining_time = p[i].burst_time;
    }

    printf("\nMasukkan Time Quantum: ");
    scanf("%d", &quantum);

    roundRobin(p, n, quantum);

    // Hitung waktu tunggu & turnaround
    float totalWT = 0, totalTAT = 0;
    for (int i = 0; i < n; i++) {
        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
        p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;
        totalWT += p[i].waiting_time;
        totalTAT += p[i].turnaround_time;
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].arrival_time, p[i].burst_time,
            p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
    }

    printf("\nAverage Waiting Time: %.2f\n", totalWT / n);
    printf("Average Turnaround Time: %.2f\n", totalTAT / n);

    return 0;
}

```

```
putrafernanda@DESKTOP-TTI x + v
putrafernanda@DESKTOP-TTKUOI1:~$ nano eksperimen3case.c
putrafernanda@DESKTOP-TTKUOI1:~$ gcc eksperimen3case.c -o eksperimen3case
putrafernanda@DESKTOP-TTKUOI1:~$ ./eksperimen3case
Masukkan jumlah proses: 4

Proses 1
Arrival Time: 0
Burst Time: 24

Proses 2
Arrival Time: 1
Burst Time: 3

Proses 3
Arrival Time: 2
Burst Time: 3

Proses 4
Arrival Time: 3
Burst Time: 6

Masukkan Time Quantum: 3

===== Gantt Chart =====
| P1 | P2 | P3 | P4 | P1 | P4 | P1 | P1 | P1 | P1 | P1 | P1 |
PID  AT  BT  CT  TAT  WT
P1   0   24  36  36  12
P2   1    3   6   5   2
P3   2    3   9   7   4
P4   3    6  18  15   9

Average Waiting Time: 6.75
Average Turnaround Time: 15.75
putrafernanda@DESKTOP-TTKUOI1:~$ |
```

## Test case 2

```
putrafernanda@DESKTOP-TTI x + v
Average Waiting Time: 6.75
Average Turnaround Time: 15.75
putrafernanda@DESKTOP-TTKUOI1:~$ ./eksperimen3case
Masukkan jumlah proses: 5

Proses 1
Arrival Time: 0
Burst Time: 10

Proses 2
Arrival Time: 0
Burst Time: 5

Proses 3
Arrival Time: 0
Burst Time: 8

Proses 4
Arrival Time: 0
Burst Time: 12

Proses 5
Arrival Time: 0
Burst Time: 6

Masukkan Time Quantum: 4

===== Gantt Chart =====
| P1 | P2 | P3 | P4 | P5 | P1 | P2 | P3 | P4 | P5 | P1 | P4 |
PID  AT  BT  CT  TAT  WT
P1   0  10  37  37  27
P2   0   5  25  25  20
P3   0   8  29  29  21
P4   0  12  41  41  29
P5   0   6  35  35  29

Average Waiting Time: 25.20
Average Turnaround Time: 33.40
putrafernanda@DESKTOP-TTKUOI1:~$ |
```

### Test case 3

```
putrafernanda@DESKTOP-TT1 x + v
Average Waiting Time: 25.20
Average Turnaround Time: 33.40
putrafernanda@DESKTOP-TT1:~$ ./eksperimen3case
Masukkan jumlah proses: 5

Proses 1
Arrival Time: 0
Burst Time: 8

Proses 2
Arrival Time: 2
Burst Time: 4

Proses 3
Arrival Time: 4
Burst Time: 9

Proses 4
Arrival Time: 6
Burst Time: 5

Proses 5
Arrival Time: 8
Burst Time: 3

Masukkan Time Quantum: 2

===== Gantt Chart =====
| P1 | P2 | P3 | P4 | P5 | P1 | P2 | P3 | P4 | P5 | P1 | P3 | P4 | P1 | P3 | P3 |

PID  AT  BT  CT  TAT  WT
P1   0   8  26  26  18
P2   2   4  14  12   8
P3   4   9  29  25  16
P4   6   5  24  18  13
P5   8   3  19  11   8

Average Waiting Time: 12.60
Average Turnaround Time: 18.40
putrafernanda@DESKTOP-TT1:~$
```

## Tugas 5: Analisis Eksperimen perbandingan algoritma

Setelah menjalankan program dengan berbagai test case, jawab pertanyaan berikut:

### 1. Analisis Performa

- a. Dalam kondisi apa FCFS memberikan hasil terbaik?

FCFS (First Come First Served) bekerja paling baik ketika semua proses memiliki waktu burst yang hampir sama dan datang secara berurutan tanpa perbedaan arrival time yang ekstrem.

- b. Mengapa SJF hampir selalu memberikan Average WT terkecil?

Karena SJF (Shortest Job First) selalu mengeksekusi proses dengan burst time paling kecil terlebih dahulu, sehingga proses singkat tidak perlu menunggu proses panjang selesai.

- c. Apa trade-off dari Round Robin?

Round Robin memberikan pembagian waktu yang adil (fairness) antar proses, tetapi:

- Terjadi banyak context switch, yang meningkatkan overhead CPU.
- Jika quantum terlalu kecil, maka efisiensi menurun.
- Jika quantum terlalu besar, maka perilakunya menjadi mirip FCFS (tidak lagi adil).

## 2. Context Switching

a. Algoritma mana yang paling banyak melakukan context switch?

Round Robin memiliki jumlah context switch paling banyak karena CPU terus berpindah antar proses berdasarkan quantum waktu.

b. Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin? Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi?

Semakin kecil quantum, semakin banyak context switch dan lebih responsif, tapi dengan biaya efisiensi CPU. Semakin besar quantum, semakin sedikit context switch, tapi respons time menurun.

## 3. Fairness

a. Algoritma mana yang paling "adil" untuk semua proses? • Proses mana yang paling dirugikan di algoritma FCFS?

Round Robin paling adil karena setiap proses mendapat jatah CPU dengan durasi quantum yang sama.

b. Apakah ada kemungkinan starvation di SJF?

Proses dengan burst time pendek yang datang terlambat.

Karena proses panjang yang datang lebih dulu akan memonopoli CPU, menyebabkan proses pendek menunggu lama (*convoy effect*).

## 4. Real World Application

a. Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?

Algoritma yang cocok: Round Robin (RR)

b. Untuk render video (task besar), algoritma mana yang cocok?

Algoritma yang cocok: FCFS atau SJF (Shortest Job First)

c. Untuk OS desktop (interactive), algoritma mana yang cocok?

Algoritma yang cocok: Round Robin (dengan prioritas dinamis)